White Paper: Secure and Scalable High-Performance Cryptography

Author: Roberto Avanzi

Version v0.1.0, 2026-04-27: Draft

Executive Summary

Cloud computing has become the foundation of modern digital infrastructure, supporting a vast range of services from financial systems to artificial intelligence workloads. Client devices are the primary means of accessing these services. As a result, both cloud and client systems are attractive targets for malicious actors. This reality makes cryptography essential: it protects data, secures communication, and enables trust in distributed and pervasive environments.

However, cryptography faces an old, fundamental challenge: systems can either protect sensitive material effectively or process it efficiently, but rarely both. While cloud services demand responsive, high-speed cryptographic operations, common solutions such as accelerated CPU instructions provide performance at the cost of fundamental security risks, even in architectures that hide keys from software.

To resolve this tension, we propose ACE, the *Atomic Cryptographic Extension*. ACE is an architectural solution for integration into instruction set architectures. It is built around the notion of a *Cryptographic Context*, a protected data structure that encapsulates cryptographic secrets, usage policies, and state information. ACE instructions use these Cryptographic Contexts in place of actual key values when performing cryptographic operations. The latter are executed *atomically*, in the sense that while cryptographic modes and protocols are split into multiple calls to primitives, the primitives themselves are not split into individual rounds.

ACE offers an interface from the CPU to the cryptographic hardware, not too dissimilar from a closely-coupled, or *attached* accelerator block. However, the instructions, the data structures, and the interface have been taylored to the the needs of cryptographic operations and with a focus on security, correctness, and agility.

This allows systems to perform cryptographic operations at high speed while ensuring correctness and integrity.

To fully realize this vision in cloud-scale heterogeneous computing environments, we also propose that the industry converge on common standards for representing Cryptographic Contexts and guidelines for managing and using them.

The Need High-Performance, Secure Cryptography

Cryptography has evolved from a specialized tool for securing a narrow range of applications into a fundament of modern computing. The internet, mobile devices, and cloud computing have dramatically expanded its scope. Today, cryptography protects most data flows, including storage—increasingly often both at the file system and block level—, memory, network traffic, and increasingly application logic itself.

This shift affects emerging workloads as well: Large language models are encrypted at rest and decrypted only during execution; distributed systems rely on encryption and authentication across services; and confidential computing protects sensitive data during processing. In these scenarios, cryptography is no longer an occasional operation: it is an integral part of the steady-state execution of any system.

As a result, its performance is no longer a secondary concern. Any inefficiency directly impacts latency, throughput, and ultimately the cost and scalability of services and applications. Simultaneously, any weakness in it threatens the assets of both cloud service providers and their users. Resolving this tension between performance and security remains a significant challenge.

The Limits of Existing Approaches

— Specialized Environments

Historically, the computational cost of cryptographic operations and limitations of general-purpose CPUs led to the development of separate hardware cryptographic accelerators, for instance the Sun Cryptographic Accelerator PCI boards or the IBM 4758/4764/4765 Cryptographic Modules. These later evolved into hardened modules which are often used in high-assurance environments (Attridge, 2021) and into secure processors for embedded applications.

While such components provide strong protection of cryptographic secrets, they come with expensive setup and invocation overheads. As general-purpose CPUs became faster, these overheads sometimes negated the performance benefits of specialized hardware, especially for smaller payloads. This drove a shift: when low-latency and responsiveness matter, cryptographic operations are now frequently implemented directly in firmware or software. However, such implementations expose keys and intermediate values in memory, enabling serious attacks such as memory disclosure vulnerabilities (van der Veen et al., 2012; Lou et al., 2022) or cold boot attacks (Halderman et al., 2009; Yitbarek et al., 2017).

To protect cryptographic keys and sensitive data, systems have added isolation layers in the form of trusted execution environments, such as TrustZone (Alves & Felton, 2004), or secure enclaves, such as Intel SGX (Intel, 2015), often paired with memory encryption and authentication (Gueron, 2016) or private on-chip memory. While these mechanisms may be more efficient than specialized hardware and reduce exposure of secrets to general-purpose software, they still carry a performance penalty from switches to protected environments and overhead of additional countermeasures, which sometimes include disabling processor caches. Other factors include disruption of execution flow, increased cache pressure (if caches are not disabled), and increased memory access latency.

In large-scale cloud systems performing millions of cryptographic operations per second, such costs accumulate quickly.

— General Purpose Computation Environments

Implementing cryptography in general-purpose execution environments with acceleration instructions such as VIA's PadLock (VIA, 2005) and Intel's AES-NI (Gueron, 2012) addresses the performance problem. However, as noted earlier, these features expose keys to memory or must be used in trusted software environments to prevent exposure. Thus, a desirable feature is the ability to protect a key without having to perform a context switch each time it is required.

Intel's KeyLocker (Intel, 2020) and IBM's Protected Key (IBM, 2025) functionalities move in the right direction by allowing encrypted keys to be used instead of actual values. The hardware decrypts the keys internally when needed. The values of the keys may be set by software or hardware but using the encrypted keys never exposes their clear values to software environments.

However, this still leaves a more subtle but equally serious problem unaddressed: *hiding the keys does not necessarily ensure that the desired cryptographic guarantees cannot be violated.*

Complex modes of operation require confidentiality and integrity not only of the main key but also of any derived key and internal state such as intermediate values of the authentication tag. Exposure of these values can compromise modes entirely, allowing attackers to violate confidentiality or forge messages regardless of key recovery. This applies to modes such as XTS (Dworkin, 2010), GCM-SIV (Gueron et al., 2019) and OCB (Krovetz & Rogaway, 2021), as well as upcoming NIST-standardized algorithms whose security proofs depend on the secrecy of all derived keys and values.

Additionally, some cryptographic accelerators accept both a secret key and an initialization vector to prevent misuse. Key-hiding-only ISAs lack this capability.

Protecting only the initial key while leaving the rest of the computation visible is comparable to locking the front door and closing the windows while leaving the shutters of the latter up. While the window panes may be made of highly resistant glass, this creates a false sense of security without addressing the full scope of the problem.

Finally, the current practice of defining new instructions for each cryptographic algorithm is unsustainable. As state actors plan new cryptographic standards and the European Union is even in the process of compiling a potentially large catalogue of allowed algorithms, this approach may lead to instruction proliferation. In modern RISC architectures with limited encoding space, each new instruction requires a strong justification.

Therefore, we need a solution that limits encoding space usage, ideally using a constant number of instructions regardless of how many algorithms are supported.

A Comprehensive Approach

We now describe our approach to designing an ISA extension for cryptographic operations that provides not only key secrecy and integrity, but also protects the state machine of cryptographic algorithms while invoking the underlying cryptographic primitives in a fine-grained way.

— Cryptographic Contexts

What is needed is not just a more secure way to store and use keys, but a more secure way to think about cryptographic execution overall and a way to abstract cryptographic algorithm specifics from the instruction set. Our concept of *Cryptographic Context* provides such an abstraction.

A Cryptographic Context extends the concept of an encrypted key by including all the information required for a cryptographic operation to be realized as a sequence of smaller operations: the key material, the algorithm that can be used with the key material, any additional values used by the algorithm, the internal state of the latter, and *Metadata* governing how it may be used. Besides the allowed algorithm, the Metadata also includes usage policies which optionally define usage constraints based on the hardware and software environment, security state, boot cycle, or other policies. Instead of treating these elements as separate pieces of information that move through the system, the context binds them together into a single protected entity. Cryptographic Contexts allow us to treat cryptographic algorithms as stateful, encapsulated objects, to which information is passed in a structured manner rather than sequences of stateless primitive invocations.

While software can *provision* key(s) and Metadata into Cryptographic Context, the latter, once provisioned, cannot be inspected, copied, or modified arbitrarily by software. Instead, it can only be used through well-defined operations that strictly follows the policies allowed by the architecture and by the algorithm. In this sense, a cryptographic context behaves like a secure object whose internal structure is opaque but whose functionality is precisely controlled.

Cryptographic Contexts reside in dedicated architectural storage, which we call *Context Registers*. These are not addressable as normal memory and not directly readable by software.

— ACE Instructions and the ACE Unit

All ACE operations reference Context Registers rather than the key values. This limits the visibility of key material to the provisioning phase only, which can be performed by trusted entities. All successive cryptographic computations remain inside a protected hardware domain while avoiding additional switches to protected environments, and only the final results are visible to software.

While cryptographic modes and protocols are split into calls to primitives, the primitives themselves are not split into individual rounds. It is important that these operations are executed atomically, such as full AES block encryptions and decryptions, steps in modes of operation, and hash absorption rounds. This prevents leakage from revealing the outputs of intermediate steps: even knowing the input and output to five rounds of an AES encryption will significantly facilitate key recovery (Bouillaguet et al., 2012; Bar-On et al., 2018), therefore we must always compute the full operation.

The two most important ACE instructions are the *execution* of a unit of the algorithm (e.g., encrypting or decrypting a block of data, absorbing a block of data into a hash), and the request for a *change in the state* of the algorithm (e.g., moving from initialization to data absorption and to finalization in a hash, or moving from the processing of associated data to the processing of data to be encrypted/decrypted and finally to a tag generation or verification in an AEAD algorithm).

The ACE instructions function like messages to a closely coupled, i.e., “attached”, hardware unit, which can be, in principle, shared amongst several cores, even heterogeneous ones, while keeping

a separate architectural state for each hardware thread.

— Lifecycle of a Cryptographic Context

A Cryptographic Context can be exported from a Context Register to memory as a Sealed Cryptographic Context (SCC), and the latter can be reimported into a Context Register. A SCC is an encrypted and authenticated representation of the contents of the Context Register, and it preserves all the policies defined in the metadata. This mechanism uses a *Sealing Key* which can be securely reconfigured. It is used to enable secure context switching, and the reconfigurability of the Sealing Key allows for cryptographic separation of CCs between process domains, and also enables to migrate keys between devices along with their VMs, by migrating the Sealing Key alongside them.

Advantages

By design, ACE eliminates many traditional attack surfaces. Keys and derived values never appear in a form visible to software, reducing the risk of leakage through memory inspection, memory disclosure vulnerabilities, or side-channel attacks. The integrity of a CC is preserved, ensuring that both the data and the policies governing its use remain intact. At the same time, the architecture can enforce constraints on how a context is used, preventing certain classes of misuse: The fact that a state is carefully maintained ensures that the instructions for processing and state changes execute in a strictly prescribed manner, even if the process is interrupted for context switching, the Context Registers are exported and then reimported before resuming the process. This significantly reduces the ways an algorithm can be misused or compromised.

There is in principle no limit to the complexity of the algorithm that can be represented in a Cryptographic Context, for instance even complex integrated public key encryption algorithms can be defined by using a very rich state and multiple execution phases.

Since the choice of the algorithm is embedded in the Metadata, the architecture can offer also offer masked implementations of the same primitives and algorithms to protect against side-channel attacks. Software using Cryptographic Contexts requires no modification to support these protected operations.

Equally important, this approach enables high-speed cryptographic operations, by eliminating the need to move data into and out of isolated environments.

ACE is particularly valuable for cloud providers. It allows cryptography to be integrated into performance-critical paths—such as storage, networking, and AI inference—minimizing bottlenecks and complexity.

Call for Action

While Cryptographic Contexts offer a powerful foundation, their full potential can only be realized through industry-wide adoption. This starts with an important ISA adopting the concept, but ideally it requires that more ISAs implement the concept in a coordinated manner.

Today’s cloud environments are heterogeneous, combining multiple processor architectures and

hardware platforms. Key providers and consumers may operate on different ISAs, and without a common specification, protected cryptographic material cannot move securely across these boundaries. This fragmentation limits flexibility and increases operational complexity.

To realize this vision, the industry must establish two critical standards:

- First, ISAs must implement consistent functionality for Cryptographic Contexts, using identical data formats for Context Register initialization and Sealed Cryptographic Contexts.
- Second, there must be a shared specification of Cryptographic Context semantics, including algorithm behavior, state evolution, and usage constraints. This ensures that contexts created in one environment function correctly and securely in another.

Conclusion

The growing importance of cryptography in cloud computing has exposed the limitations of existing approaches. Systems that rely on isolation achieve strong security but struggle to meet performance demands, while those that prioritize efficiency often leave critical gaps in protection.

The concept of Cryptographic Contexts, as defined in our proposal ACE, offers a way forward. They enable software to perform high-throughput cryptographic operations while ensuring that keys and their sensitive derived values remain protected, as well as ensuring their algorithmic correctness.

Realizing this vision will start with one ISA, but unleashing its full potential requires an industry-wide effort towards standardization—in formats, semantics, and interfaces–to allow a seamless flow of cryptographic information across heterogeneous systems. All without forcing a compromise between efficiency and security. The fact that ACE can support additional primitives and modes through a flexible interface that does not require the addition of new instructions for each new algorithm allows better integration with multiple ISAs, since only the ACE specification needs to be updated. A separate consortium of industry participants can work to define common formats and semantics for ACE.

Bibliography

Alves, T., & Felton, D. (2004). Trustzone: Integrated hardware and software security – ARM white paper. *Information Quarterly*, *3*, 18–24.

Attridge, J. (2021). *An Overview of Hardware Security Modules*. The Escal Institute of Advanced Technologies, Inc. d/b/a SANS Institute. https://www.sans.org/white-papers/757

Bar-On, A., Dunkelman, O., Keller, N., Ronen, E., & Shamir, A. (2018). Improved Key Recovery Attacks on Reduced-Round AES with Practical Data and Memory Complexities. In H. Shacham & A. Boldyreva (Eds.), *Advances in Cryptology - CRYPTO 2018 - 38th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 19-23, 2018, Proceedings, Part II* (pp. 185–212). Springer. https://doi.org/10.1007/978-3-319-96881-0_7

Bouillaguet, C., Derbez, P., Dunkelman, O., Fouque, P.-A., Keller, N., & Rijmen, V. (2012). Low-Data Complexity Attacks on AES. *IEEE Trans. Inf. Theory*, *58*(11), 7002–7017. https://doi.org/10.1109/TIT.2012.2207880

Dworkin, M. (2010). Recommendation for Block Cipher Modes of Operation: The XTS-AES Mode for Confidentiality on Storage Devices. In *Federal information processing standards (FIPS), National Institute of Standards and Technology, Gaithersburg, MD*. https://csrc.nist.gov/pubs/sp/800/38/e/final

Gueron, S. (2012). *Intel® Advanced Encryption Standard (Intel® AES) Instructions Set - Rev 3.01*. https://www.intel.com/content/www/us/en/developer/articles/tool/intel-advanced-encryption-standard-aes-instructions-set.html

Gueron, S. (2016). A Memory Encryption Engine Suitable for General Purpose Processors. *IACR Cryptology EPrint Archive*. http://eprint.iacr.org/2016/204

Gueron, S., Langley, A., & Lindell, Y. (2019). *AES-GCM-SIV: Nonce Misuse-Resistant Authenticated Encryption*. http://www.ietf.org/rfc/rfc8452.txt

Halderman, J. A., Schoen, S. D., Heninger, N., Clarkson, W., Paul, W., Calandrino, J. A., Feldman, A. J., Appelbaum, J., & Felten, E. W. (2009). Lest we remember: cold-boot attacks on encryption keys. *Commun. ACM*, *52*(5), 91–98. https://doi.org/10.1145/1506409.1506429

IBM. (2025). *z/Architecture Principles of Operation, 15th Edition*. https://www.ibm.com/docs/en/module_1678991624569/pdf/SA22-7832-14.pdf

Intel. (2015). *Intel® Software Guard Extensions*. https://software.intel.com/en-us/isa-extensions/intel-sgx

Intel. (2020). *Intel® Key Locker Specification*. https://www.intel.com/content/www/us/en/content-details/671438/intel-key-locker-specification.html

Krovetz, T., & Rogaway, P. (2021). The Design and Evolution of OCB. *J. Cryptol.*, *34*(4), 36. https://doi.org/10.1007/S00145-021-09399-8

Lou, X., Zhang, T., Jiang, J., & Zhang, Y. (2022). A Survey of Microarchitectural Side-channel Vulnerabilities, Attacks, and Defenses in Cryptography. *ACM Comput. Surv.*, *54*(6), 122:1–122:37.

https://doi.org/10.1145/3456629

van der Veen, V., dutt-Sharma, N., Cavallaro, L., & Bos, H. (2012). Memory Errors: The Past, the Present, and the Future. In D. Balzarotti, S. J. Stolfo, & M. Cova (Eds.), *Research in Attacks, Intrusions, and Defenses - 15th International Symposium, RAID 2012, Amsterdam, The Netherlands, September 12-14, 2012. Proceedings* (pp. 86–106). Springer. https://doi.org/10.1007/978-3-642-33338-5_5

VIA. (2005). *VIA PadLock Programming Guide*. https://web.archive.org/web/20100526054140/http://linux.via.com.tw/support/beginDownload.action?eleid=181&fid=261

Yitbarek, S. F., Aga, M. T., Das, R., & Austin, T. M. (2017). Cold Boot Attacks are Still Hot: Security Analysis of Memory Scramblers in Modern Processors. *2017 IEEE International Symposium on High Performance Computer Architecture, HPCA 2017, Austin, TX, USA, February 4-8, 2017*, 313–324. https://doi.org/10.1109/HPCA.2017.10